Neural Surrogate Modeling for Snake Robot Locomotion:

Faster Reinforcement Learning via Learned Cosserat Rod Dynamics

[Author Name]

Abstract

[Placeholder: Write abstract after all results are in.]

Contents

1 Introduction **3**

2 Background **3**

2.1 Cosserat Rod Dynamics 3

2.2 Resistive Force Theory Contact Model 4

2.3 Central Pattern Generator Actuation 4

2.4 PyElastica Simulator 5

3 Related Work **5**

3.1 Neural Surrogates for Soft Robots 5

3.2 Reinforcement Learning with Learned Simulators 6

4 Methods **7**

4.1 Data Collection Pipeline 7

4.2 State and Action Representation 8

4.3 Per-Element CPG Phase Encoding 8

4.4 Surrogate Model Architecture 9

4.5 Training Procedure 9

5 Experiments and Results **10**

5.1 Surrogate Accuracy 10

5.2 RL Training with Surrogate 10

5.3 Comparison with Direct Elastica RL Baseline 11

6 Discussion **11**

6.1 Physics Calibration Challenges 11

6.2 Surrogate Architecture Experiments 12

6.3 Data Collection Pipeline Challenges 12

7 Conclusion **13**

1 Introduction

[Placeholder: Introduction: problem statement, contribution summary, paper roadmap.]

2 Background

2.1 Cosserat Rod Dynamics

The body of the snake robot is modeled as a Cosserat rod — a one-dimensional elastic continuum that supports shear, extension, bending, and torsion along its arc length (Till et al., 2019). Unlike rigid-body chain models, the Cosserat formulation captures the distributed compliance of the rod, making it well-suited for soft and continuum robots whose deformation is governed by material elasticity rather than joint constraints. The governing equations, stated per unit arc length s, are the linear and angular momentum balances:

$$\rho A \frac{\partial^2 \mathbf{x}}{\partial t^2} = \frac{\partial \mathbf{F}}{\partial s} + \mathbf{f}_{\text{ext}}, \tag{1}$$

$$\rho \mathbf{I} \frac{\partial^2 \mathbf{d}}{\partial t^2} = \frac{\partial \mathbf{M}}{\partial s} + \mathbf{m}_{\text{ext}} + \mathbf{x}' \times \mathbf{F}, \tag{2}$$

where $\mathbf{x}(s,t) \in \mathbb{R}^3$ is the rod centerline position, $\mathbf{d}(s,t)$ is the material director frame encoding local orientation, $\mathbf{F}$ is the internal force (shear and tension, from the constitutive law), $\mathbf{M}$ is the internal moment (bending and torsion), $\mathbf{f}_{\text{ext}}$ denotes external contact forces from ground interaction, and $\mathbf{m}_{\text{ext}}$ represents the muscle moments generated by the CPG actuator. The term ρA is the linear mass density and $\rho \mathbf{I}$ is the rotational inertia per unit length.

Although the full Cosserat model operates in three dimensions, snake locomotion in this work is constrained to the horizontal plane: all z-components of position and velocity are verified to be identically zero, so the system reduces to a two-dimensional projection without loss of physical content.

After spatial discretization into $N = 21$ nodes ($N_e = 20$ elements), the Cosserat PDEs (eqs. (1) and (2)) become a first-order ordinary differential equation (ODE) system that is integrated forward in time by a symplectic Euler stepper. The resulting discrete state vector $\mathbf{s}_t \in \mathbb{R}^{124}$ is laid out as follows:

Table 1: Layout of the 124-dimensional rod state vector $\mathbf{s}_t$.

Index slice	Variable	Count	Physical meaning
`[0:21]`	x_i	21	Node x-positions (m)
`[21:42]`	y_i	21	Node y-positions (m)
`[42:63]`	$\dot{x}_i$	21	Node x-velocities ($\mathrm{m\,s^{-1}}$)
`[63:84]`	$\dot{y}_i$	21	Node y-velocities ($\mathrm{m\,s^{-1}}$)
`[84:104]`	ψ_e	20	Element yaw $= \arctan 2(\mathrm{tang}_y, \mathrm{tang}_x)$ (rad)
`[104:124]`	$\omega_{z,e}$	20	Element angular velocity about z-axis ($\mathrm{rad\,s^{-1}}$)

This 124-dimensional state is the Markov state used for both surrogate training and RL policy input. The lower-dimensional observation space (14 floats based on FFT compression of node positions) that the policy receives is not Markov — identical observations can yield different next-states depending on the hidden rod configuration and velocity history — so the surrogate must operate on the full 124-dimensional rod state to produce correct multi-step rollouts.

2.2 Resistive Force Theory Contact Model

Ground contact forces are modeled using Resistive Force Theory (RFT), a slender-body approximation in which the contact force at each rod element depends only on the local velocity components resolved into directions tangent and normal to the element's centerline (Till et al., 2019). This formulation avoids the computational cost of solving a full contact optimization at each substep, while capturing the anisotropic friction that is the physical mechanism of serpentine propulsion.

The tangential and normal drag forces per unit length acting on element e are:

$$f_t = -c_t\, v_t, \tag{3}$$

$$f_n = -c_n\, v_n, \tag{4}$$

where v_t and v_n are the velocity components tangent and normal to the element, respectively, and c_t, c_n are anisotropic drag coefficients. In this work we use $c_t = 0.01$ and $c_n = 0.05$, giving a normal-to-tangential ratio of five. The anisotropy $c_n > c_t$ is the essential physical mechanism that enables net forward motion from a traveling curvature wave: lateral slip is resisted more strongly than axial slip, so the reaction forces from the ground produce a net thrust in the forward direction.

2.3 Central Pattern Generator Actuation

The controller generates a traveling curvature wave κ^{target} across the rod body; an elastic restoring force within PyElastica then drives each element toward its local target curvature, producing the characteristic serpentine gait. This Central Pattern Generator (CPG) approach parameterizes the entire gait with a small number of physically meaningful variables, which facilitates both learning and analysis.

The target curvature at element i is:

$$\kappa_i^{\text{target}} = A \sin\big(2\pi k \cdot s_i - 2\pi f \cdot t + \phi_0\big) + b, \tag{5}$$

where $s_i \in [0, 1]$ is the normalized arc-length position of element i, A is the amplitude (rad m^{-1}), k is the wave number (wavelengths per body length), f is the oscillation frequency (Hz), ϕ_0 is the initial phase offset (rad), and b is a constant turn bias (rad m^{-1}). The five-dimensional action vector $\mathbf{a} = (A, f, k, \phi_0, b)$ parameterizes this wave, with physical ranges $A \in [0, 5]$, $f \in [0.5, 3.0]$ Hz, $k \in [0.5, 3.5]$, $\phi_0 \in [0, 2\pi]$, and $b \in [-2, 2]$. The RL policy outputs a normalized action $\mathbf{a} \in [-1, 1]^5$ that is linearly mapped to these physical ranges at each control step.

2.4 PyElastica Simulator

PyElastica (Naughton et al., 2021) is an open-source Python framework for compliant mechanics simulation based on Cosserat rod theory. It implements the discretized equations of motion described in section 2.1 using a symplectic Euler integrator, which provides numerical stability for the stiff elastic rod dynamics without the computational cost of implicit methods. Each RL control step (of duration Δt_{ctrl}) runs n_{sub} internal physics substeps, defining the ground-truth transition operator $T : (\mathbf{s}_t, \mathbf{a}_t) \mapsto \mathbf{s}_{t+1}$.

Each evaluation of T requires approximately 46 ms when running 16 parallel environment workers on a 48-core CPU server, yielding a throughput of roughly 350 environment steps per second. At this rate, a PPO training run requiring 2×10^6 environment interactions takes approximately 36 hours of wall-clock time. This computational bottleneck motivates replacing T with a neural surrogate model $f_\theta \approx T$ that can be evaluated in a single GPU forward pass ($\sim$0.001 ms per step), enabling massive parallelization and a projected throughput improvement of four to five orders of magnitude.

3 Related Work

3.1 Neural Surrogates for Soft Robots

Approximating the dynamics of elastic and soft robots with neural networks has attracted substantial research attention as a means of bypassing the prohibitive cost of first-principles simulation. The strongest result in this direction for Cosserat rod dynamics comes from Stolzle et al. (2025), who propose a domain-decoupled physics-informed neural network (DD-PINN) that achieves a 44,000$\times$ speedup over numerical Cosserat rod integration while maintaining end-effector position errors below 3 mm (2.3% of actuator length) across the full 72-dimensional rod state. The key innovation is the decoupling of the time domain from the feed-forward network, which enables gradients to be computed in closed form rather than via expensive automatic differentiation through the ODE integrator. The resulting model enables nonlinear model-predictive control (MPC) at 70 Hz on a GPU — a capability that is simply infeasible with direct numerical integration.

Building on this foundation, Hsieh et al. (2024) adopt a hybrid approach in KNODE-Cosserat: a simplified first-principles Cosserat rod model serves as the backbone, while a neural ODE residual term corrects the systematic error between the simplified physics and real hardware behavior. Crucially, the training data for KNODE-Cosserat is collected from PyElastica simulations — a pipeline directly analogous to the one used in this work. The resulting hybrid model achieves a 58.7% accuracy improvement over the physics-only model on long-horizon trajectories, demonstrating that even a coarse physics prior substantially reduces the burden on the neural correction term.

The most directly relevant prior work to this paper is SoRoLEX (Berdica et al., 2024), which trains an LSTM forward dynamics model on soft robot simulator data and uses the learned model as a surrogate environment for PPO training with JAX-based GPU parallelization. The central pipeline — collect trajectories from a high-fidelity simulator, train a neural dynamics model, and run large-scale RL in the learned environment — is identical to the approach taken here. Where this work differs is in the choice of architecture (MLP rather than LSTM) and in the prediction target (the full 124-dimensional Cosserat rod state rather than a task-space observation), and in the explicit use of a multi-step rollout loss to limit autoregressive drift during RL rollouts.

In a related thread, Author et al. (2025) demonstrate that physics-informed neural networks for articulated soft robots can achieve a 467× speedup over accurate first-principles models while enabling MPC at 47 Hz, and require as little as a single training dataset — illustrating the data efficiency advantages that come from incorporating physical structure into the surrogate architecture.

Taken together, these results establish that neural surrogates for Cosserat rod and soft robot dynamics are feasible, practical, and capable of sustaining the multi-step rollout accuracy required for closed-loop control. This work follows the data-driven approach of SoRoLEX but with an MLP predicting the full Cosserat rod state (124 dimensions), trained with both single-step MSE and multi-step rollout loss to reduce the autoregressive error accumulation that poses the main challenge for long-horizon RL training.

3.2 Reinforcement Learning with Learned Simulators

The idea of replacing expensive environment interactions with a learned dynamics model dates to Dyna (Sutton, 1990, as cited in Janner et al., 2019), and has recently been placed on a rigorous theoretical and empirical footing by a series of influential results. Janner et al. (2019) show in Model-Based Policy Optimization (MBPO) that an ensemble of learned dynamics models can be used to generate short synthetic rollouts from real environment states, and that policy optimization on these rollouts achieves model-free asymptotic performance with approximately five times less real-environment interaction. The central insight of MBPO — that short-horizon model predictions limit error accumulation while still providing useful policy gradient signal — directly motivates the use of a surrogate environment for RL in this work.

The world model paradigm extends this approach by learning compact latent representations of environment dynamics alongside the policy. Janner et al. (DreamerV3, 2019)

demonstrate that a recurrent state-space model trained entirely from sensory observations can sustain imagination-based RL across 150 diverse tasks without task-specific tuning, illustrating that surrogate quality is the primary bottleneck for downstream policy performance. In the context of the present work, the accuracy of the Cosserat rod surrogate — quantified in Phase 4 validation — directly determines the quality of policies trained in the surrogate environment. Our Phase 8 comparison with a direct PyElastica RL baseline isolates the policy gap attributable to surrogate modeling error.

Graph-structured alternatives to MLP surrogates have also been explored for deformable body simulation. Sanchez-Gonzalez et al. (2020) introduce a particle-based graph network simulator (GNS) that treats simulation as message passing over a particle graph, achieving up to 1000$\times$ speedups for fluid, rigid, and deformable material simulation. Pfaff et al. (2021) extend this approach to mesh-based simulation with MeshGraphNets, which operate directly on unstructured simulation meshes and generalize across different mesh resolutions. For the snake robot considered in this work, however, the rod is discretized into only 20 elements — a sufficiently small graph that the added architectural complexity of a GNN is not warranted, and a dense MLP achieves comparable expressive power at a fraction of the implementation cost. Graph-structured surrogates would become advantageous if the problem were extended to rods with significantly more nodes or to multi-rod assemblies interacting through contact.

4 Methods

4.1 Data Collection Pipeline

Training data for the surrogate model is collected by running 16 independent worker processes, each maintaining its own PyElastica Cosserat rod simulation. Each worker samples actions from the 5-dimensional CPG action space using a Sobol quasi-random sequence, which provides better coverage of the action space than pseudo-random sampling by distributing points more uniformly across all dimensions. To populate diverse regions of the state space, 30% of episodes begin with a state perturbation: random position offsets are applied to node positions, and angular velocity noise is injected into the rod elements. Within each episode, 50% of actions are drawn uniformly at random while the remaining 50% follow the current Sobol sequence; this balance between exploration and structured coverage prevents over-sampling of any particular action regime.

The V2 collection format, introduced to support the per-element phase encoding described in section 4.3, uses a checkpoint-style storage scheme. Each worker executes a single action four times consecutively, saving the rod state before execution and after each of the four sub-steps, yielding five boundary states per collection run. This produces four overlapping training pairs (s_t, a_t, s_{t+1}) from each run without discarding episode boundary transitions. The target dataset size is approximately 25 GB, corresponding to roughly 50 M transition pairs.

During validation of the initial dataset (V1), the surrogate was found to predict angular

velocity poorly, particularly for states with large $|\omega_z|$. Analysis revealed that the initial perturbation standard deviation of $\sigma_\omega = 0.05\,\text{rad/s}$ was far below the operational CPG range of 1–10 rad/s, leaving this region severely underrepresented. For the V2 dataset, σ_ω was increased to 1.5 rad/s, providing the coverage needed for accurate angular velocity prediction. The full discussion of this coverage gap and its implications is given in section 6.3.

4.2 State and Action Representation

The full 124-dimensional rod state vector $\mathbf{s}_t \in \mathbb{R}^{124}$ defined in section 2.1 and table 1 is used as the Markov state for both surrogate training and RL policy input. As noted in section 2.1, snake locomotion in this work is constrained to the horizontal plane: all z-components of position and velocity are identically zero, so the three-dimensional Cosserat state reduces to this 124-dimensional 2D projection without loss of physical content. The 124 floats comprise 21 node x-positions, 21 node y-positions, 21 node x-velocities, 21 node y-velocities, 20 element yaw angles, and 20 element angular velocities about the z-axis.

The 5-dimensional action vector $\mathbf{a}_t = (A, f, k, \phi_0, b) \in [-1, 1]^5$ is defined in section 2.3 and linearly mapped to the physical ranges $A \in [0, 5]\,\text{rad}\,\text{m}^{-1}$, $f \in [0.5, 3.0]\,\text{Hz}$, $k \in [0.5, 3.5]$, $\phi_0 \in [0, 2\pi]\,\text{rad}$, and $b \in [-2, 2]\,\text{rad}\,\text{m}^{-1}$ at each control step. The normalized action $\mathbf{a}_t$ is passed directly to the surrogate as part of its input vector.

4.3 Per-Element CPG Phase Encoding

The target curvature at element e depends on both the global oscillation phase $2\pi ft$ and the element's arc-length position s_e through the serpenoid wave (eq. (5)). As a consequence, elements at opposite ends of the rod can be at completely opposite phases of the curvature wave at the same instant in time. The V1 surrogate used a global phase encoding $[\sin(2\pi ft), \cos(2\pi ft)] \in \mathbb{R}^2$, appended once to the input. With only the global phase available, the model had to reconstruct the per-element phase offset $2\pi k \cdot s_e$ implicitly through shared MLP weights across all 20 elements simultaneously — a significant implicit learning burden that contributed to poor angular velocity predictions ($R^2 = 0.23$; see section 6.3).

The V2 encoding provides the per-element phase to the model explicitly. For element $e \in \{1, \ldots, N_e\}$, the effective time t_e at that element's arc-length position s_e is computed as $t_e = k \cdot s_e / f$, giving an element-specific phase triplet:

$$\boldsymbol{\phi}_e = \left(\sin(2\pi f \cdot t_e),\ \cos(2\pi f \cdot t_e),\ \kappa_e^{\text{actual}}\right) \tag{6}$$

where κ_e^{actual} is the current actual curvature of element e (available in the rod state). The sine and cosine terms give the model the exact phase of the serpenoid wave at each element's location; the actual curvature provides the control error $(\kappa_e^{\text{target}} - \kappa_e^{\text{actual}})$ that drives the elastic restoring force. Stacking $\boldsymbol{\phi}_e$ for all $N_e = 20$ elements yields a 60-dimensional per-element phase feature vector. The phase features are computed on-the-fly from the stored action and episode start time t_{start} during dataset loading, so the batch files store only the rod states, actions, and t_{start} without any pre-computed phase data.

The full model input concatenates the z-score normalized rod state $\bar{\mathbf{s}}_t$, the normalized action $\mathbf{a}_t$, and the per-element phase features:

$$\mathbf{z}_t = \left[\bar{\mathbf{s}}_t \parallel \mathbf{a}_t \parallel \boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_{N_e}\right] \in \mathbb{R}^{189} \tag{7}$$

where $\bar{\mathbf{s}}_t = (\mathbf{s}_t - \mu_s)/\sigma_s$ and μ_s, σ_s are the training-set per-feature mean and standard deviation. The 189 input dimensions decompose as $124 + 5 + 60$.

4.4 Surrogate Model Architecture

The surrogate is a feedforward MLP that approximates the ground-truth transition operator T from section 2.4:

$$f_\theta : \mathbf{z}_t \mapsto \overline{\Delta\mathbf{s}}, \qquad \hat{\mathbf{s}}_{t+1} = \mathbf{s}_t + \sigma_\Delta \odot f_\theta(\mathbf{z}_t) + \mu_\Delta \tag{8}$$

where $\mu_\Delta, \sigma_\Delta \in \mathbb{R}^{124}$ are the training-set per-feature mean and standard deviation of the state delta $(\mathbf{s}_{t+1} - \mathbf{s}_t)$, computed once before training begins. The model predicts the *normalized* delta $\overline{\Delta\mathbf{s}}$; the actual next state is recovered via the affine reconstruction in eq. (8).

Delta prediction provides two practical advantages over predicting the absolute next state. First, per-step state changes are small relative to absolute state values — position deltas are typically on the order of centimeters per 0.5 s control step while absolute positions can span a meter — so the target distribution is more concentrated and easier to fit. Second, when $f_\theta(\mathbf{z}_t) \approx 0$ the reconstruction gives $\hat{\mathbf{s}}_{t+1} \approx \mathbf{s}_t + \mu_\Delta$, which is a reasonable default (the mean empirical delta), acting as an implicit residual connection.

The network architecture is:

$$\mathbb{R}^{189} \xrightarrow{\text{Linear}} [\text{Linear}(512) \to \text{LayerNorm} \to \text{SiLU}]^{\times 3} \xrightarrow{\text{Linear}} \mathbb{R}^{124} \tag{9}$$

The output layer is initialized to zero so that the model starts by predicting zero delta (i.e., the mean empirical transition), providing stable early-training behavior. Total trainable parameters: approximately 659K. LayerNorm is applied after each linear transformation rather than after the activation to stabilize the training dynamics with the SiLU nonlinearity, which can produce large activations for large positive inputs.

4.5 Training Procedure

Training proceeds in two phases that introduce the rollout loss gradually to avoid instability during early optimization.

Phase 1 (epochs 1–20): single-step MSE loss. The model is trained to minimize the mean squared error on the normalized delta:

$$\mathcal{L}_{\text{single}} = \frac{1}{N} \sum_{i=1}^{N} \left\| f_\theta(\mathbf{z}_t^{(i)}) - \overline{\Delta\mathbf{s}}_{\text{true}}^{(i)} \right\|^2 \tag{10}$$

This phase trains the model to make accurate one-step predictions before introducing the

more expensive rollout computation.

Phase 2 (epoch 20 onward): combined loss with rollout regularization. An 8-step autoregressive rollout loss is added to penalize error accumulation over multi-step predictions:

$$\mathcal{L} = \mathcal{L}_{\text{single}} + \lambda_r \cdot \mathcal{L}_{\text{rollout}} \tag{11}$$

$$\mathcal{L}_{\text{rollout}} = \frac{1}{L} \sum_{t=0}^{L-1} \gamma^t \left\| \hat{\mathbf{s}}_{t+1} - \mathbf{s}_{t+1}^{\text{true}} \right\|^2 \tag{12}$$

with rollout weight $\lambda_r = 0.1$, rollout length $L = 8$, and discount $\gamma = 1$ (no temporal discounting). Each rollout step uses the model's own prediction $\hat{\mathbf{s}}_{t+1}$ as input for the next step; gradients are back-propagated through the entire unrolled trajectory via BPTT. The rollout loss directly penalizes the drift between surrogate rollouts and ground-truth PyElastica trajectories, which is the primary failure mode when the surrogate is used as a substitute environment for RL.

Density-weighted sampling. Sobol action sampling produces a non-uniform distribution over the state space, with states near the center of the action space overrepresented. To compensate, each training sample is assigned an importance weight proportional to the inverse of its estimated local density:

$$w^{(i)} = \frac{1}{\hat{p}(\mathbf{c}^{(i)})}, \qquad \mathbf{c}^{(i)} = \left[\bar{x}_{\text{CoM}},\ \bar{y}_{\text{CoM}},\ \|\dot{\mathbf{x}}\|,\ |\bar{\omega}_z|\right] \tag{13}$$

where $\mathbf{c}^{(i)} \in \mathbb{R}^4$ is a 4-dimensional summary feature capturing center-of-mass position, speed, and angular velocity magnitude. The density $\hat{p}$ is estimated from a joint histogram with 20 bins per dimension, and weights are clipped at $10\times$ the mean weight to prevent any single rare sample from dominating the gradient.

Training hyperparameters. The best configuration from an architecture sweep (described in section 6.2) used: learning rate 10^{-3}, Adam optimizer with default momentum parameters, batch size 4096, and early stopping on the validation MSE. All experiments were logged to Weights & Biases for tracking and comparison.

5 Experiments and Results

5.1 Surrogate Accuracy

[Placeholder: Phase 4/5/8 results here. Surrogate validation against held-out PyElastica trajectories: 1-step MSE, multi-step rollout RMSE, per-state-component accuracy (ω_z poor prediction issue and fix).]

5.2 RL Training with Surrogate

[Placeholder: Phase 4/5/8 results here. PPO training on surrogate environment: learning curves, wall-clock time comparison, policy performance on surrogate vs real Elastica environment.]

5.3 Comparison with Direct Elastica RL Baseline

[Placeholder: Phase 4/5/8 results here. Head-to-head comparison: surrogate RL vs direct Elastica RL (Phase 8 baseline). Sample efficiency, wall-clock time, final policy quality, transfer gap.]

6 Discussion

6.1 Physics Calibration Challenges

Establishing a reliably locomoting Cosserat rod simulation required resolving three categories of physics calibration errors. Each issue independently prevented stable serpentine locomotion and would have rendered any surrogate trained on the resulting data physically meaningless. These findings illustrate a general principle: a surrogate model of an incorrectly calibrated physics engine will faithfully replicate the engine's errors — producing a learned model that is fast but wrong.

Rod radius and bending stiffness. The initial rod radius was set to $r = 0.001$ m, matching a plausible physical snake scale. However, bending stiffness scales as $EI \propto r^4$, where E is Young's modulus and $I = \pi r^4/4$ is the second moment of area. At $r = 0.001$ m, a Young's modulus of $E = 2 \times 10^6$ Pa (the initial value) gave $EI \approx 1.6 \times 10^{-6}$ N·m^2 — small enough that the rod was effectively limp and produced near-zero locomotion velocity. Increasing E to resolve this produced a qualitatively different failure: at $E \geq 10^6$ Pa the rod snapped violently toward the target curvature, with actual curvatures reaching ± 100 rad/m (compared to targets of ± 2.5 rad/m) and node velocities exceeding 178 m/s, causing the simulation to diverge. The resolution came from a grid search over (E, r) pairs, which identified $r = 0.02$ m with $E = 10^5$ Pa as producing stable serpentine motion. Increasing the radius by a factor of 20 increases I by a factor of 160,000, so even though the Young's modulus was simultaneously reduced by a factor of 20, the net bending stiffness EI increased sufficiently to drive meaningful rod deformation at physically plausible velocities. This calibration illustrates a fundamental modeling challenge: the parameter values required to produce correct physical behavior in a Cosserat rod simulator need not correspond to the physical material parameters of the modeled object, particularly when the simulator's internal length scales and integration timesteps span orders of magnitude different from the physical system.

Serpenoid wave direction sign error. The CPG actuation module contained a sign error in the temporal term of the serpenoid curvature equation. The initial implementation computed $\kappa(s,t) = A\sin(k \cdot s - \omega t + \phi)$, which produces a curvature wave with phase velocity $+\omega/k$ in the $+s$ direction — propagation from tail to head (anteriorly). Anisotropic RFT friction generates net forward thrust only when the curvature wave propagates posteriorly (head to tail), so anterior propagation causes the snake to swim backward. The correct expression is $\kappa(s,t) = A\sin(k \cdot s + \omega t + \phi)$, with phase velocity $-\omega/k$ in the $+s$ direction. This bug was not detected during initial simulator construction because the training environment computed curvatures inline rather than calling the CPG module's export method, so the erroneous code path was exercised only when the module was called externally. The error

was identified only after observing consistent negative forward-velocity rewards across multiple training runs, which prompted a careful comparison of the CPG wave equation against the serpentine gait literature. This experience motivates explicit unit tests that verify the direction of wave propagation for a known CPG configuration, rather than relying on training metrics to surface physics errors.

RFT friction coefficient tuning. Anisotropic drag coefficients derived from published values for soil-crawling snakes ($c_n/c_t \approx 10$) produced excessive lateral drift on the flat-surface simulation substrate used here. A grid search over the ratio c_n/c_t found that $c_t = 0.01$ and $c_n = 0.05$ (ratio 5:1) yielded stable forward propulsion at the 94:1 forward-to-lateral velocity ratio needed for consistent locomotion across the full range of CPG parameters. The discrepancy from published values reflects the different contact mechanics of dry-surface locomotion (no soil deformation, no interlocking with granular media), confirming that substrate-specific friction re-tuning is necessary when the simulation contact model is transferred across physical regimes.

6.2 Surrogate Architecture Experiments

[Placeholder: Surrogate architecture experiments: rollout loss variants, residual MLP vs baseline MLP, history window experiments, architecture sweep comparison findings (Phase 03 Plans 03–05).]

6.3 Data Collection Pipeline Challenges

Building a reliable parallel data collection system for the surrogate model required resolving four engineering challenges. Each arose from the interaction between PyElastica's Numba-compiled physics kernel and the Python multiprocessing infrastructure used to achieve parallelism.

Numba thread pool deadlock. PyElastica's Cosserat rod solver is implemented as a collection of Numba JIT-compiled functions. When new worker processes are spawned via the `forkserver` start method, they inherit Python module state but not the Numba thread pool runtime. On the first physics evaluation in a new worker process, Numba re-initializes its internal thread pool (defaulting to 48 threads on the 48-core machine). Under heavy parallel load — 16 workers each spawning 48 Numba threads yields 768 total threads competing for 48 cores — a low-probability but non-zero thread pool deadlock occurred at episode reset boundaries. The hang was silent: no crash, no exception, no progress. The fix was straightforward: setting `NUMBA_NUM_THREADS=1` before any PyElastica import eliminated Numba's internal thread parallelism entirely. This is the correct trade-off for this workload: each PyElastica instance operates on a single 20-element rod, so Numba's intra-function parallelism provides no benefit, while process-level parallelism across 16 workers provides a 16× throughput improvement.

Stall detection false positives during initialization. To maintain data collection throughput across long multi-hour runs, a health monitor tracks the data output rate of each worker and respawns any worker that produces no new transitions for an extended interval. During initial deployment, this monitor caused a kill-respawn cycle that prevented any data from

being collected: all 16 workers were flagged as stalled and respawned within 60 seconds of startup. The root cause was that PyElastica worker initialization is itself expensive: each worker must fork via `forkserver`, import NumPy and PyElastica, construct the Cosserat rod with material properties, initialize the RFT friction model, and warm the Numba JIT cache by running a first simulation step. Under load on a 48-core machine, this initialization sequence takes more than 60 seconds — exactly equal to the stall detection threshold. The resolution was to condition the stall counter on the worker having produced at least one transition: zero-progress polls during initialization are not counted as stalls. This grace-period pattern — wait for first output before enforcing liveness — is more robust than simply increasing the timeout, which would reduce detection sensitivity for genuine post-initialization stalls.

Parallel scaling bottleneck. Increasing the number of workers beyond 16 produced diminishing throughput returns, and beyond approximately 32 workers the throughput plateaued rather than continuing to grow. The bottleneck was memory bandwidth rather than CPU time. PyElastica's Cosserat rod solver performs many small array operations (position updates, internal force computation, geometric computations) that are memory-bound: the CPU arithmetic finishes before the next array is fetched from memory. At high worker counts, the shared L3 cache (30 MB per socket) is exceeded by the aggregate working sets of all workers, increasing cache miss rates and saturating the memory bus. The optimal configuration for the 48-core machine was 16 workers — approximately 8 per socket — which keeps L3 contention manageable and achieves near-linear scaling relative to single-threaded throughput. The surrogate model approach ultimately sidesteps this bottleneck: once the dataset is collected, RL training runs entirely on GPU using the surrogate, at throughputs orders of magnitude higher than any CPU-bound physics simulation.

Angular velocity coverage gap. Initial analysis of the V1 dataset revealed that the surrogate's predictions for element angular velocity (ω_z) were qualitatively poor, with mean $R^2 = 0.23$ and individual elements below zero. Examination of the dataset's marginal distributions showed that the state perturbation standard deviation for angular velocity of $\sigma_\omega = 0.05$ rad/s was far smaller than the operational CPG range of 1–10 rad/s: the training data contained almost no transitions with large $|\omega_z|$, so the model had not been exposed to the states where accurate angular velocity prediction matters most. For the V2 dataset, σ_ω was increased to 1.5 rad/s, improving coverage of the full operational range. This case illustrates a general issue with surrogate training data: if the data collection distribution does not cover the regions of state space that the downstream RL policy will encounter, the surrogate will be systematically inaccurate precisely where accuracy is needed most.

7 Conclusion

[Placeholder: Write conclusion after all results are in.]

References

A. Author et al. Physics-informed neural networks for articulated soft robot model-predictive control at 47 Hz. *arXiv preprint arXiv:2502.01916*, 2025. URL https://arxiv.org/abs/2502.01916. 467× speedup; MPC at 47 Hz; as little as 1 training dataset.

Uljad Berdica et al. SoRoLEX: Learned environments for soft robot reinforcement learning. *arXiv preprint arXiv:2410.18519*, 2024. URL https://arxiv.org/abs/2410.18519. LSTM surrogate for Cosserat rod sim; order-of-magnitude speedup for RL.

Tsung-Wei Hsieh et al. KNODE-Cosserat: Neural ODE correction of cosserat rod dynamics. *arXiv preprint arXiv:2408.07776*, 2024. URL https://arxiv.org/abs/2408.07776. 58.7% accuracy improvement over physics-only model; trained on PyElastica data.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1906.08253.

Noel Naughton, Jiarui Sun, Arman Tekinalp, Tejaswin Parthasarathy, Girish Chowdhary, and Mattia Gazzola. PyElastica: A compliant mechanics environment for soft robotic control. *IEEE Robotics and Automation Letters*, 6(2):3389–3396, 2021. doi: 10.1109/LRA.2021.3063698.

Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2010.03409.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 8459–8468, 2020. URL https://arxiv.org/abs/2002.09405.

Maximilian Stolzle et al. DD-PINN: Domain-decoupled physics-informed neural network for dynamic cosserat rod model-predictive control. *arXiv preprint arXiv:2508.12681*, 2025. URL https://arxiv.org/abs/2508.12681. 44,000× speedup over numerical integration; MPC at 70 Hz on GPU.

John Till, Vincent Aloi, and Caleb Rucker. Real-time dynamics of soft and continuum robots based on Cosserat rod models. *The International Journal of Robotics Research*, 38(6):723–746, 2019. doi: 10.1177/0278364919842269.